

MCP Privilege Gateway Technical Summary v3

Overview

PingOne Privilege MCP Gateway is positioned as an inline MCP security gateway for governed access to MCP servers, tools, prompts, and resources. Internally, it is being used primarily for workforce and personal-assistant style use cases where just-in-time access, credential protection, approval workflows, time-bound authorization, and auditability matter.

At a high level, deployment consists of:

- Creating the MCPGW OIDC application
- Preparing the gateway host or container environment
- Configuring `pingone.env` and TLS certificates
- Registering the gateway in PingOne Privilege
- Creating the MCP Server application
- Associating the mesh cluster / gateway
- Defining policies for tools, prompts, resources, and approvals
- Validating authentication, routing, and activity logs

Core Configuration

Host prerequisites

- PingOne tenant with PingOne Privilege enabled
- Access to the PingOne admin console
- At least one backend MCP server
- One or more MCP clients
- A host or VM capable of running Docker
- A DNS name for the gateway

Ports and network

- `8623/TCP` for MCP traffic
- `8690/TCP` for gateway-to-gateway / mesh communication
- Standard management access such as `22`, `443`, and optionally `3389`
- Outbound access to required PingOne endpoints and backend MCP servers

Environment file

Typical `pingone.env` values include:

None

```
SERVER_URL=https://<mcpgw-dns>:<port>
OIDC_CLIENT_ID=<id>
OIDC_CLIENT_SECRET=<secret>
OIDC_AUTH_URL=https://auth.<pingurl>pingone.com/<env>/as/authorize
OIDC_TOKEN_URL=https://auth.<pingurl>pingone.com/<env>/as/token
OIDC_USER_URL=https://auth.<pingurl>pingone.com/<env>/as/userinfo
OIDC_SCOPES=openid profile email
```

TLS

Certificates are typically placed at:

- `/var/lib/procyon/ssl/mcpgw-cert.pem`
- `/var/lib/procyon/ssl/mcpgw-key.pem`

The certificate CN/SAN should match the deployed gateway DNS name.

Deeper Internal Findings

- Privilege MCP Gateway is internally positioned as the preferred option when the problem is governed MCP access near the user or agent, especially for approvals, JIT access, credential protection, and auditability.
- A useful internal mental model is: **Privilege guards the agent/client side**, while **PingGateway safeguards the MCP server/resource side**.
- The endpoint controller / local agent is **not required** for the Privilege MCP Gateway in agentless mode.
- Agentless mode uses PingOne SSO rather than hardware-bound device identity.
- For CLI access in agentless scenarios, internal guidance points to **PCLI** rather than the full endpoint agent.
- Internal discussion references a dual-SSO chain in some patterns: one SSO flow from the MCP client to Privilege GW, and another from Privilege GW to the downstream MCP server.
- Current backend MCP authentication handling appears somewhat immature in practice, with discussion of hard-coded tokens or headers in some current deployments.
- A concrete example discussed internally is Splunk MCP, where the desired pattern is for the gateway to inject the Splunk token rather than expose it directly to the user.
- Operationally, deployment is still fairly manual, especially for IaaS / container-hosted rollouts.

Known Deployment Gotchas

- **SERVER_URL** appears to be required in practice. Internal troubleshooting suggests missing **SERVER_URL** in **pingone.env** can break expected gateway behavior.
- Mount **pingone.env** as a file, not just loose environment variables. Teams discussed mounting it directly at **/var/lib/procyon/config/pingone.env**.
- **Kubernetes ingress often needs 443 -> 8623 mapping**. This is an important implementation detail for customer deployments.
- **Some MCP client flows may need the /mcp path**. Internal troubleshooting noted cases where the bare frontend URL returned **200 OK**, but adding **/mcp** triggered the expected OIDC redirect and flow behavior.
- **OIDC env values may influence .well-known/oauth-protected-resource behavior**. This helps explain why the OIDC config must be correct even early in testing.
- **DNS, TLS, and network reachability remain common failure points**. These were recurring themes in deployment and troubleshooting discussions.
- **UI / product workflow maturity is still evolving**. Internal discussion suggested deployment and configuration were still more manual than fully productized in some stages.

Privilege vs PingGateway

Area	PingOne Privilege MCP Gateway	PingGateway MCP Security Gateway
Primary role	Governed access to MCP tools and servers	Token and request enforcement in front of backend services
Logical placement	Near the user, agent, or MCP client	Near the MCP server or protected resource
Best fit	Workforce AI, personal assistant, JIT access, approvals, credential shielding	Resource-server enforcement, OAuth validation, introspection, request transformation
Auth / secret pattern	Can inject vaulted or JIT credentials so users and agents do not directly hold them	Validates and transforms inbound auth context before passing traffic onward

Area	PingOne Privilege MCP Gateway	PingGateway MCP Security Gateway
Policy focus	User/group access, time-bound access, approvals, allowed tools/prompts/resources	API gateway-style policy and token enforcement
Operational model	Closely tied to Privilege governance flows	More aligned to programmable gateway / resource protection patterns

Recommended Additions for Field and Deployment Docs

1. Add an explicit **Known Deployment Gotchas** section covering `SERVER_URL`, env file mounting, ingress mapping, `/mcp` path behavior, and `.well-known` dependencies.
2. Add a **Privilege vs PingGateway** comparison so teams do not position the wrong product for the use case.
3. Add an **agentless mode note** clarifying that the endpoint agent is optional for this pattern.
4. Add a **backend authentication caveat** noting that some integrations may still rely on headers or static tokens today.
5. Add a **Kubernetes implementation note** for ingress, file mounting, and certificate handling.

Bottom Line

The public setup guidance is directionally correct, but internal material adds important operational nuance: agentless SSO is real, the endpoint agent is optional for this use case, `SERVER_URL` and env-file mounting matter, Kubernetes ingress details matter, and the cleanest architecture split is **Privilege for governed access near the agent** versus **PingGateway for token and request enforcement near the server**.